Avathon Track B - Ravi Shankar Prasad
Predicting Pricing-Review Questions from Deal Notes

Ravi Shankar Prasad — Track B (SLM Fine-Tuning), Scenario S2

1. Problem and approach

We sell diagnostics instruments into complex deals. The instrument goes in at near-zero margin; the money comes from reagents over the life of the lease. Every deal is a different combination of moving parts — discount requested, lease length, reagent volume, line margins, competitor presence, trade-ins — and no two look alike. A pricing manager reads the deal note and, from experience, knows which of those parts is the problem here, and asks about it. That judgement is intuition built over hundreds of deals, not a written rule. Today that judgement reaches the rep as a round of questions over email. A pricing manager carries a high volume of deals, so every round sits in a queue before it is answered, and most deals take several rounds. That back-and-forth dominates the clock: **deal turnaround runs close to 90 days**, almost none of it spent thinking — it is spent waiting for the next email.

The system predicts, from a free-text deal note, **which questions a pricing manager will ask** — drawn from a 14-lever taxonomy (13 of which fire on this dataset), typically 1 to 13 per deal. The rep answers them before submitting. The target is cycle time, not headcount.

Why this needs AI rather than a form. The cheap fix is to make reps fill in structured fields so a rules engine can test them. That fails for the reason it always fails: reps write deal notes as prose, in a hurry, in their own style, and a mandatory form gets filled with placeholders or routed around. The input is unstructured and will stay unstructured, so there are no fields for a rules engine to read. What is needed is something that reads free text and decides which of many interacting conditions apply — reading *plus* judgement. And the judgement itself has to be **learned from the deals rather than transcribed from a manual**: the regularities a pricing manager applies are latent across thousands of past reviews and written down nowhere, by anyone. No conventional system can acquire a rule that has never been stated — only something that learns from examples can. That is the AI opportunity, and the payoff is cycle time.

Track B follows from that. Facts (margin floors, price lists) belong in a prompt or a retrieval index; the judgement does not. Retrieval returns instances and has no step that synthesises a policy from them. Fine-tuning is that step.

2. Algorithm decisions and alternatives rejected

Fine-tuning over RAG. Retrieval would be right if the policy were written down. It isn't. Retrieval remains right for the volatile facts — thresholds change quarterly and editing a config beats retraining.

Fine-tuning over prompt engineering — tested, not assumed. The untrained rows below *are* prompt engineering: all 14 questions supplied in the prompt. A prompted 3B asks 12 questions where 5 are needed, and **61% of what it writes matches none of the 14 it was just given.** Three structural objections beyond accuracy: a real policy is ~100 interacting conditions, so the prompt grows without bound and is re-sent per deal; long prompts dilute attention, so the rule that decides *this* deal gets no more weight than the 99 that don't; and a wrong question cannot be traced to a cause — the fix is rewording and re-running, which silently changes behaviour elsewhere. A fine-tune is not self-explaining either, but failures are localisable: the behaviour came from a training set I can query, and every change is re-scored on a fixed held-out set.

Few-shot rejected too — the strongest version of the prompting case. Supplying k solved deals in the prompt teaches format and tone quickly, but not decision boundaries: the model copies the examples rather than applying the condition behind them, so novel feature combinations fall outside anything it can imitate. My base rows are the *generous* version of this test — they receive the rules themselves rather than examples of rules applied, strictly more information than few-shot — and still go 56–82% off-list. Few-shot also pays per call forever, and ~100 interacting conditions cannot be demonstrated in a handful of cases. **I did not run a separate few-shot baseline**, so this is an argument from the zero-shot result, not a measurement of its own.

A script fixes the truth; the LLM only writes words. The obvious shortcut is to let one model invent a deal and another read the numbers back out. The problem is that nothing can then check the second model — whatever it reads becomes the answer by default. My script sets the values first, so the extractor can be scored against them: GPT-4o gets stated fields right **87.9%** of the time, but for fields the note never mentions it makes one up **16.4%** of the time. One field in six would be fiction, so the rule engine runs on the script's known values and never on extracted ones.

Rules select the questions; the model only phrases them. A general model's guess would encode an industry it doesn't work in. The rules were drafted from one real 14-turn review and corrected by hand; replaying them against that transcript reproduces **13 of its 14 questions in order**.

Disjoint surface, not a random split. Test deals use author voices and question wordings absent from training (40 wordings per lever: 30 train, 10 test). Machine-checked every run — an earlier generator leaked 70% of test questions and was caught before any GPU time. Final: **0 shared questions, 0 shared deals.**

Model selection. Qwen2.5 0.5B/1.5B/3B and Llama 3.2 1B/3B, sizes matched so scale is tested within a family and family across matched sizes. Rejected Phi-3-medium and Llama 70B (over the 13B cap), Gemma/Phi-mini as second family (sizes don't align, confounding family with scale), and a 7B fine-tune (5 hours to answer what the base 7B answers in ten minutes).

QLoRA rejected; 16-bit base throughout. QLoRA squeezes the frozen base to 4-bit so a larger model fits the same card. That trade only pays when memory is the binding constraint — above roughly 7B. At 0.5B–3B with frozen weights and small adapters, everything fits 16GB comfortably, so 4-bit would buy memory I already have and charge quantisation error in every base weight to do it. It also complicates serving: an adapter trained against 4-bit weights does not merge cleanly into 16-bit ones, so deployment needs a dequantise-merge-requantise step and a re-validation. Training in 16-bit meant the adapter merges into exactly the weights it was trained against — measured at 2.48GB served, well inside any card. Had the brief forced a 7B, QLoRA would have been mandatory rather than optional, since 14GB of 16-bit weights leaves no room to train. I also rejected distillation and pruning: the premise is that a *small* model suffices, so the right move is to start small, not shrink something large.

LoRA rank 16, all seven projections. Full fine-tuning rejected — won't fit 16GB and there's no reason to move 3B weights for a behavioural change. Attention-only rejected: module *placement* matters more than rank, and the feed-forward block holds most of the parameters. DoRA rejected — its advantage concentrates at rank 4–8 and costs time per step. Prefix tuning rejected: it steers attention but can't change how evidence is combined. **Trade-offs accepted:** LoRA can't absorb new world knowledge (not needed — this is behavioural); rank 16 caps complexity (beneficial on 700 examples); one recipe across all five models means none is at its own optimum (deliberate — per-model sweeps would make the scaling comparison meaningless). Adapters merge at the end, so inference cost is zero.

Loss masked to the question list only, so no gradient is spent learning to write deal notes.

3. Technical execution

One entry point (`python run.py`), seven stages, each writing its artifact to disk so any stage can be re-run independently:

1. **Generate** — a script invents 1,000 deals, drawing and deriving every field arithmetically. These values are the answer key.
2. **Write** — GPT-4o renders each deal as a sales-voice note, omitting some facts. Four voices for train/val; two different voices reserved for test.

3. **Extract** — side branch, not on the training path: measure whether the fields can be read back out (87.9% on stated, 16.4% invention on omitted).
4. **Select** — the rule engine fires levers against the script's *own* values, one condition per lever. 5,074 questions over 1,000 deals, mean 5.07. This list is the label.
5. **Phrase** — 40 natural wordings per lever, 560 in total; wordings 1–30 go to training, 31–40 to test. Split 700/100/200.
6. **Train** — five models, identical recipe, LoRA adapters only.
7. **Evaluate** — ten runs (five base, five tuned) plus two no-model baselines, same 200 test deals, same code.

Business rules are isolated in one module so they can be read and corrected by someone who does not read Python.

Seeds, PYTHONHASHSEED and the cuBLAS workspace are pinned before any import; every artifact's SHA-256 goes to a manifest. **The manifest produced on Windows matches the one produced on Linux.** All LLM calls are cached on a hash of (model, messages, temperature), so the dataset regenerates byte-identically without an API key.

Correctness details that matter in this track: chat templates applied through the tokenizer, never hand-written; adapter targets detected by scanning the model rather than hard-coded; training arguments checked against the installed library signature at runtime, with remaps logged; loss compared only within a family, since different tokenizers make per-token loss incomparable (372 vs 379 tokens for identical examples).

Bugs that changed results: rule triggers tested only exhausted levers, not answered ones (reviews died after 2 questions); payback derived from revenue instead of the reverse (76% of deals implausible, now 18%); and the leakage check compared whole lists instead of individual questions, **reporting zero while 70% leaked**. The last is the instructive one — a verification step that is itself wrong manufactures confidence, and it was caught only by testing it against a case where leakage was known to exist.

Evidence of learning, not memorisation. The test set is held out on three axes at once: unseen deals, unseen author voices, unseen question wordings. A memorising model scores near zero on the third, since every target sentence is new. It produces correct *counts* on unseen deals (4.1 against a gold mean of 4.74) where untrained models emit 8–19 regardless of input; off-list collapses on unseen phrasings; and validation loss does not track task score, which it would if surface patterns were being memorised.

4. Results and honest error analysis

200 held-out deals. Micro-F1 over the question set.

Model	Base F1	Tuned F1	Off-list base→tuned	Questions base→tuned
Llama 3.2 1B	0.319	**0.559**	68% → 6%	14.6 → 8.7
Qwen2.5 3B	0.361	0.508	61% → 14%	12.1 → 4.1
Llama 3.2 3B	0.256	0.484	56% → 8%	8.0 → 10.5
Qwen2.5 0.5B	0.314	0.471	82% → 19%	19.2 → 4.6
Qwen2.5 1.5B	0.349	0.390	62% → 24%	13.9 → 3.2

Gold mean: 4.74 questions. **Every tuned model beat every base model** — the weakest tuned score exceeds the strongest base score. What fine-tuning taught is visible in the diagnostics rather than the F1: untrained models invent 56–82% of their questions *despite having the taxonomy in the prompt*, tuned models 6–24%, and the count comes under control.

Scale is not monotonic. Qwen 0.5B beats Qwen 1.5B; Llama 1B beats Llama 3B and wins overall. Two families, same direction. With 700 examples this task is about constraint — suppressing elaboration — and larger models have more prior behaviour to suppress with the same data.

Families fail oppositely. Qwen under-asks (precision 0.58–0.60, recall 0.30–0.44); Llama over-asks (recall 0.64–0.67, precision 0.39–0.48). Llama's is the cheaper error here — a needless question costs two minutes, a missed margin question costs the deal.

Loss did not predict score. Qwen 1.5B had the lowest base validation loss (3.002, ppl 20.12) and the worst tuned F1; Qwen 3B the highest loss (3.273) and second-best F1.

Catastrophic forgetting: measured. Tuned Qwen 3B vs its base on 20 general prompts — pricing vocabulary in **0%** of general answers, mean length unchanged (565 → 549 chars), no empty outputs. Asked for a haiku, it wrote a correct original haiku. Only movement was list formatting, 35% → 45%, which is 2 of 20 prompts.

Three honest failures. (1) A no-model baseline — always ask the five most frequent questions — scores **0.620**, above every model. That is a flaw in my dataset: trigger frequencies are too skewed and the mean label too short, so a fixed guess scores well. Rebalancing is the first fix. (2) L1 appears in 200/200 gold labels yet scores 48% recall while adjacent L2 scores 1.00 with 71 false positives — the TF-IDF mapper is misfiling, not the model. (3) The base-model prompt retains wording from an earlier conversational framing and supplies no user turn, so the base-vs-tuned gap is an **upper bound**; the direction is not in doubt but the magnitude is inflated.

5. Production reality and limitations

Hallucination. The output is free text — nothing constrains it to the 14, which is why off-list rate is measurable at all. Each generated question is matched to the taxonomy with a **similarity floor**; anything below it counts as off-list rather than snapping to the nearest lever, which is what keeps the number honest. Fine-tuning is the mitigation that worked (56–82% → 6–24%). In production: the same check becomes a live filter (off-list questions withheld and logged, so the failure is a missing question rather than an invented one); manager edits become labels for a quarterly retrain; the off-list rate becomes a drift alarm; and the system runs in shadow mode against real reviews before anything reaches a rep.

Serving — measured. Tuned Llama 1B with the adapter merged back in: **2.48 GB on a Tesla T4**, median **2.9 s**, p95 4.5 s over 20 held-out deals — unbatched, no inference server, worst case. It runs on a form submission, not a chat, against a review cycle measured in days. One entry-level card serves the workload with 13.5 GB spare.

Under a tighter constraint. Had the brief forced a 7B, 14 GB of 16-bit weights on a 16 GB card leaves nothing for the KV cache — it loads and then fails on the first long note or any batching. Training would not fit at all, which is where QLoRA becomes mandatory rather than optional. For serving I would merge the adapter in **CPU memory** at 16-bit, so the 14 GB never touches the GPU, then quantise the merged model to 8-bit and serve at ~7 GB with half the card free. Not 4-bit: the T4 has no 4-bit hardware, so weights unpack on every token and it often runs slower than 8-bit, which the card supports natively. I would measure all three before committing rather than reason about it.

Output is a suggestion to the rep, never a gate on the deal. Nothing is blocked or auto-approved.

Limitations. (1) All data is synthetic; the first real step is scoring against historical deal notes. (2) A fixed guess beats every model, so the benchmark needs rebalancing. (3) Scoring depends on a TF-IDF mapper with a known confusion. (4) The base comparison used a flawed prompt. (5) The taxonomy is 14 levers from one transcript; real policy is larger. (6) Forgetting was checked on one model, not five. (7) Precision 0.39–0.60 leaves in-taxonomy-but-wrong questions; manager edits are the correction loop, not a fix.